Perfetto — ecco tutto tradotto in italiano, ordinato e pronto da seguire.
Stato attuale (breve)

- Auth (access + refresh) funzionante.
- Audit logging implementato: middleware con req.traceId, logAudit, meta salvato come JSON string, endpoint admin funzionante.
- CRUD principali (experiences, booking request) ok; booking_update ora salva traceId/ip/userAgent.
- Migrazione meta → Json rimandata a più avanti.

Piano operativo dettagliato e ordinato (da eseguire in sequenza)
Priorità 1 — Stabilità e sicurezza (immediato, ~1–2 giorni)

1. Backup e commit
   - Esegui backup DB e commit del codice. Comandi: Copy-Item .\prisma\dev.db .\prisma\dev.db.bak -Force git add -A git commit -m "Backup e checkpoint prima delle attività di stabilità"
2. Test automatici critici (integration)
   - Scrivi test per: a) auth: register/login/refresh/logout (success + fail) b) booking: create request → accept/reject → verifica audit c) admin/audits: filtri base (action, actorHostId, q)
   - Strumenti: jest o mocha + supertest.
   - Deliverable: tests/integ/*.test.js — comando npm test.
3. Controlli di sicurezza rapidi
   - Assicurati .env.example presente e segreti non nel repo.
   - Verifica che /admin/audits sia accessibile solo a role ADMIN.
   - Verifica che Audit sia read-only via API (no update/delete pubblici).

Priorità 2 — Affidabilità e pulizia (1–2 giorni)
4) Robustezza scritture audit

- Verifica tutte le chiamate a logAudit e prisma.audit.create: passare req quando disponibile.
- Hai già applicato correzioni; controlla eventuali rimanenze.

1. Rimuovi debug/temporalità
   - Sposta script temporanei in ./tools o cancellali quando non servono.
2. Indici DB
   - Aggiungi indici su Audit (createdAt, actorHostId, entityType). Modifica prisma/schema.prisma e poi: npx prisma migrate dev --name add_audit_indexes

Priorità 3 — Retention e archiviazione (2 giorni)
7) Script di archiviazione

- Implementa ./scripts/archive-audits.js:
  - seleziona audit più vecchi di X giorni, esporta JSON/CSV (o S3), poi elimina da DB.
- Esegui in cron o GitHub Actions.
- Logga l'operazione con action: audit.archive.

Priorità 4 — Osservabilità e alert (2–3 giorni)
8) Metriche e monitoring

- Esponi metriche: auditWrites/min, failedLogins/min, errors500/min (prom-client o Datadog).
- Crea alert: es. failedLogins/min > soglia.

1. Rate limiting
   - Rafforza rate limiter (login già presente): estendi ad altri endpoint critici.

Priorità 5 — CI/CD e test automatizzati (2–4 giorni)
10) Pipeline CI (GitHub Actions)
- Workflow che esegue lint, tests, prisma migrate (staging), deploy su staging.
11) Test di smoke automatizzati su staging.
Priorità 6 — Hardening & readiness produzione (3–5 giorni)
12) Secrets management (Vault / GitHub Secrets).
13) Decisione DB produzione (Postgres raccomandato vs SQLite).
14) Pooling DB, connection limits, backup/restore documentati.
Priorità 7 — Migrazioni e scalabilità (opzionale, 1–2 settimane)
15) Migrazione meta → Json (quando scelto DB)
- Script di normalizzazione (già pronto concettualmente).
- Migrerai meta String → Json e aggiornerai logAudit per salvare oggetti.
16) Valutare spostamento Audit su DB analytics (ClickHouse, Elasticsearch) se volumi alti.
Checklist operativa immediata (cose da fare ora)

- [ ] Commit e backup DB.
- [ ] Aggiungere 3 integration tests (auth, booking, audit).
- [ ] Aggiungere indici su Audit.
- [ ] Implementare script di archiviazione (bozza).
- [ ] Esportare metriche base (prom-client).
- [ ] Preparare pipeline CI con test.

Deliverable che posso generare adesso (scegli):
A) Skeleton test + 3 test di integrazione (jest + supertest)
B) Script di archiviazione (Node) + esempio GitHub Actions workflow
C) Snippet metriche (prom-client) + esempio alert rule
D) Template GitHub Actions CI (test + lint)
E) Script normalize-meta-to-json (pronto ma non eseguito)
Dimmi quale (A/B/C/D/E) generare subito e te lo preparo con file e comandi pronti.